

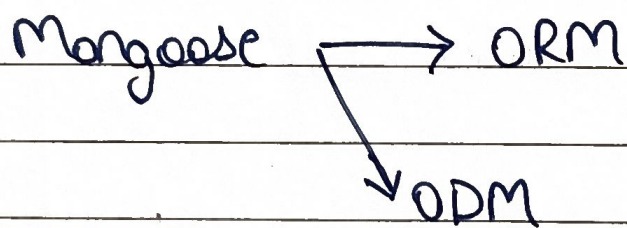
Restful / Production grade APIs

- Initialize the repo
 - install express
 - package.json

Understanding server and database connection!

- Express can talk directly with the database, then why mongoose?
- ⇒ It helps us to don't make obvious mistakes.

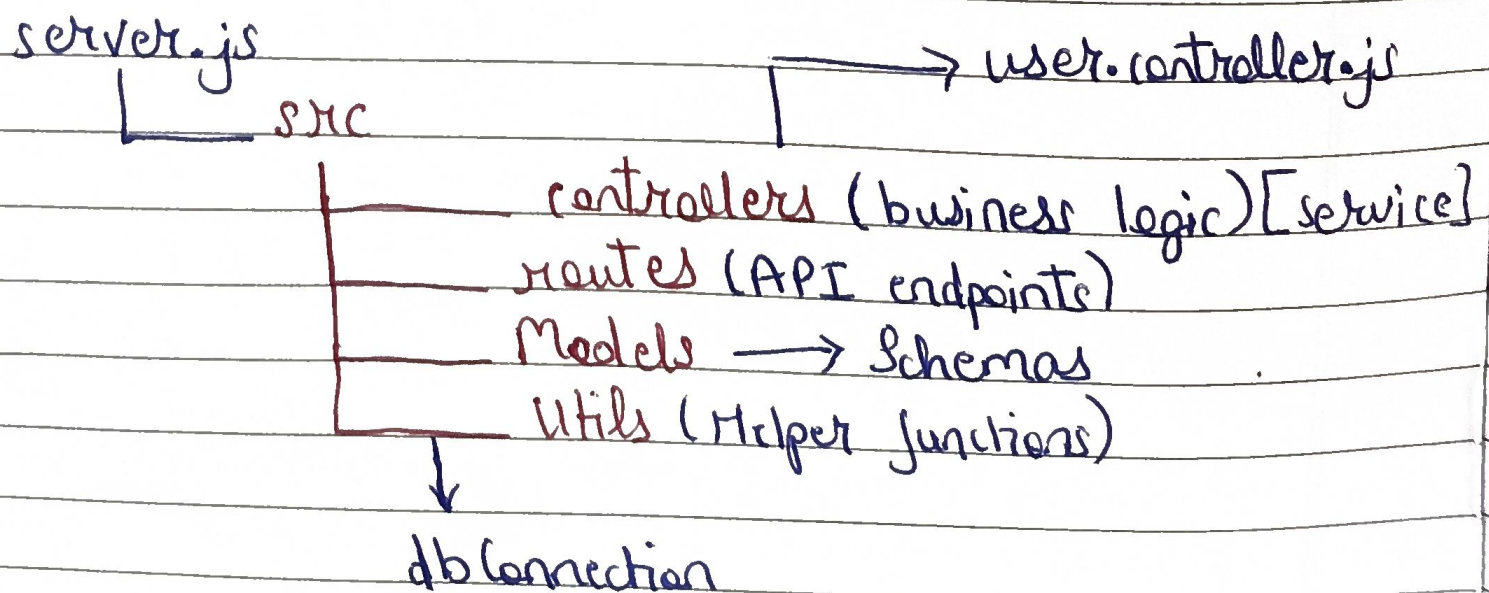
For eg:



===== folder structure =====

Most Common [MVC]

■ - Folders



→ Preparing repository for finals

- Add (.env)

~ This file is not pushed to production

mainly includes our database connection strings and keys

Industry standard $\Rightarrow$.env.example

To ignore env and node-module files
a file is created which is called

{ .gitignore }

→ Writing the application

Starting with server file

'server.js'

import 'dotenv'

package to read env file

→ add NODE-ENV = development

can use control flow to bypass ~~flow~~ logs.

→ 'app.js'

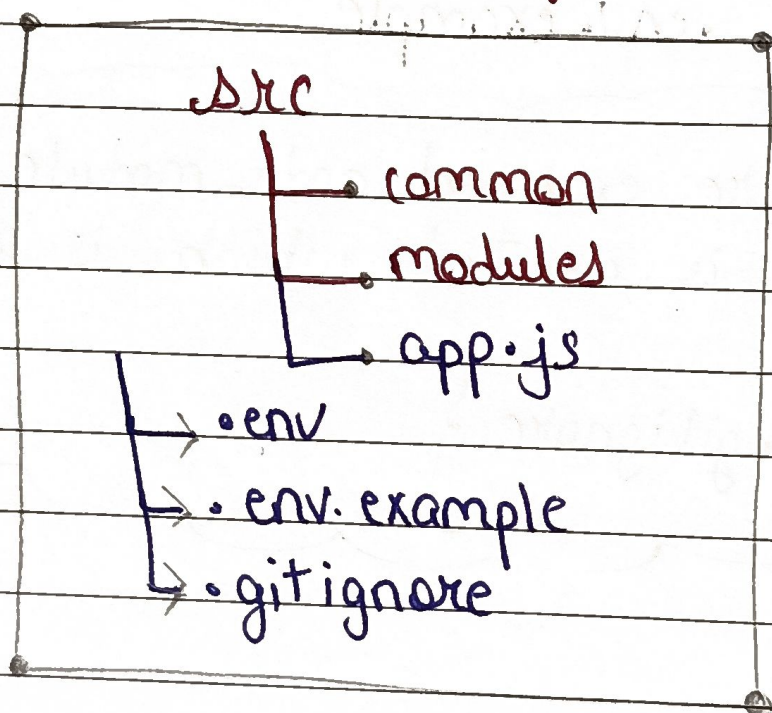
(into src directory)

- import express

- use express

- export as default

current folder structure



summary → listen to the port add catch use process.exit(1) for graceful shutdown

⇒ Install mongoose

MONGODB URI V URL

We use URI (uniform Resource Identifier) for MongoDB
(In-Build) ⚡

⇒ connecting to the database

- importing mongoose
- using async function and connect to the database with try-catch and export default

① Database is always in another continent

② Connection can be there may be not?

⇒ Utilities

· 'Api-response.js'

standard of responses

standard
structure
of
responses

→ Validations

{ zod, arktype, yup, joi, express validator }

To get the data in shape (Schema) we use a concept called DTO. (Data Transfer Object)

dto

└ base.dto.js

- import joi and install it.

create a class BaseDTO / BaseDto

destructure { early, value }

'schema = Joi.Object({

this.schema.validate

→ In Build with (joi)

abort Early ⇒ false
stripUnknown ⇒ true



cut off the
unwanted
data



give me all errors

→ return { errors, value }

Middleware (common)

└ validate-middleware.js

custom validator

↓

return a higher order function

- handle errors
- only pass sanitized data ⚠
- next()

src

└ common

└ api-response.js

└ api-error.js

└ middleware.js

└ modules

└ auth

└ dto

└ auth-controller.js

└ auth-middleware.js

└ auth-routes.js

└ auth-model.js

└ auth-service.js

Intro modules of auth (user)

→ Flow

DTO

- Database design

- id (unique)

name

email

password

isVerified

role

verificationToken

refreshToken

resetPasswordToken

resetPasswordExpiry

createdAt

updatedAt

→ timestamps : true

→ Schema using mongoose

```
'new.mongoose.Schema({})'
```

stored in key-value pairs

create ⇒ '{ }'

Homework > constants file loop -

Page 1

If does not have too pass value to client
use $\Rightarrow$

select : false

dto

register.dto.js

$\rightarrow$ Homework - password reset dto

extend dto in Basedto :
export as default

Full Flow

- routes hit at /register 'POST'
- validate (through our Base Dto)
- calls the register controller



this creates a ~~service~~ (service)

- That service holds the business logic
controller will just control the flow
- returns the api response 'created'.

Register user (service [business logic])

→ import User Schema (Model)

- simple basic register function (async)

- check user already exists

- grab the user in a variable

→ make validations on existing user

→ want to give newToken to user's email verification
and store hashedToken in database

return res(Error)

→ create user 'await'
with of the following: →

- console.log(user)

delete password

delete verificationToken

name,
email,
password,
role,

verificationToken: newToken